

Kubernetes 100 interview questions

Here is the proposed structure for the 100 Kubernetes interview questions, organized topic-wise with difficulty levels blended within each section (marked Beginner / Intermediate / Advanced per question), reflecting the kind of questions commonly asked in Indian product and service company interviews (TCS, Infosys, Wipro, product startups, and MNC DevOps/SRE roles).

Proposed Section Breakdown (100 Questions Total)

#	Section	Question Count	Difficulty Mix
1	Kubernetes Fundamentals and Architecture	12	Beginner-heavy
2	Pods, ReplicaSets, and Deployments	12	Beginner to Intermediate
3	Services, Networking, and Ingress	10	Intermediate
4	ConfigMaps, Secrets, and Environment Config	8	Beginner to Intermediate
5	Volumes and Storage (PV, PVC, StorageClass)	8	Intermediate
6	Namespaces, Labels, Selectors, and Annotations	6	Beginner

7	Scheduling, Affinity, Taints, and Tolerations	8	Intermediate to Advanced
8	RBAC, Security, and ServiceAccounts	8	Intermediate to Advanced
9	Health Checks, Probes, and Self-Healing	6	Intermediate
10	Autoscaling (HPA, VPA, Cluster Autoscaler)	6	Intermediate to Advanced
11	Helm, Kustomize, and Package Management	6	Intermediate
12	Troubleshooting and Debugging Scenarios	10	Advanced (real incident-style)
13	Advanced Architecture (etcd, API server, Controllers, CRDs, Operators)	8	Advanced

Kubernetes Interview Questions: Beginner to Advanced (100 Q&A)

Curated for real-world interview patterns commonly seen in Indian service companies (TCS, Infosys, Wipro, Capgemini, Cognizant), product startups, and MNC DevOps/SRE/Platform Engineering roles.

Section 1: Kubernetes Fundamentals and Architecture

Q1. What is Kubernetes and why is it needed? Kubernetes is an open-source container orchestration platform that automates deployment, scaling, networking, and management of containerised applications. It is needed because running containers manually at scale (hundreds of containers across many servers) becomes unmanageable, you need something to handle scheduling, self-healing, load balancing, and rolling updates automatically.

Q2. What are the main components of Kubernetes architecture? A cluster has two parts: the Control Plane (kube-apiserver, etcd, kube-scheduler, kube-controller-manager, cloud-controller-manager) which makes global decisions, and Worker Nodes (kubelet, kube-proxy, container runtime) which actually run the application containers.

Q3. What is the role of the kube-apiserver? It is the front door to the cluster, every kubectl command and every internal component talks to the API server via REST calls. It validates and processes requests, then updates the desired state in etcd. No component talks directly to etcd except the API server.

1. Authenticate
2. Authorise
3. Validate
4. Talk to ETCD -

Q4. What is etcd and why is it critical? etcd is a distributed, consistent key-value store that holds the entire cluster state (all objects: pods, deployments, secrets, configs). If etcd is lost or corrupted without backup, the cluster loses all knowledge of its own state, this is why etcd backups are a critical production practice.

Q5. What is the difference between kube-scheduler and kube-controller-manager? The kube-scheduler decides WHICH node a newly created pod should run on, based on resource availability, affinity rules, taints, etc. The kube-controller-manager runs background control loops (Node controller, Replication controller, Endpoint controller) that continuously watch the cluster and reconcile actual state with desired state.

Q6. What is a kubelet and kube-proxy? The kubelet is an agent

running on every [worker] node that talks to the API server, ensures containers described in PodSpecs are actually running and healthy. kube-proxy maintains network rules on each node so that Services can route traffic to the correct pods (traditionally via iptables or IPVS).

Q7. What is the difference between Docker and

Kubernetes? Docker is a container runtime/tool used to build and run individual containers on a single machine. Kubernetes is an orchestration layer that manages many containers across many machines, handling scheduling, scaling, networking, and failure recovery, something Docker alone does not do. Kubernetes can use containerd or CRI-O as its runtime, Docker itself is not mandatory anymore.

Q8. What is a cluster in Kubernetes? A cluster is a set of machines (nodes), a control plane and one or more worker nodes – data plane, working together as a single unit to run containerized applications.

Q9. Explain the difference between Kubernetes and Docker

Swarm. Docker Swarm is simpler to set up and has a gentler learning curve but has fewer features, weaker auto-scaling, and a smaller ecosystem. Kubernetes is more complex but offers richer scheduling, self-healing, extensibility (CRDs, Operators), and is the industry-standard choice for production at scale, which is why most companies use Kubernetes over Swarm today.

Q10. What is the difference between a Kubernetes Node and a

Pod? A Node is a physical or virtual machine (worker) that provides CPU, memory, and runs the kubelet and container runtime. A Pod is the smallest deployable unit in Kubernetes, a wrapper around one or more tightly coupled containers that share network and storage, scheduled onto a Node.

Section 2: Pods, ReplicaSets, and Deployments

Q11. What is a Pod and why you not run containers directly?

A Pod wraps one or more containers that need to share the same network namespace (same IP, same localhost) and storage volumes.

Kubernetes never schedules a bare container, it always schedules a Pod, because the Pod is the atomic unit that carries scheduling, networking, and lifecycle metadata.

Q12. Can a pod have multiple containers, when would you do that? Yes, this is called a multi-container pod, commonly used for the sidecar pattern, for example a main application container plus a logging or service-mesh proxy container (like Envoy in Istio) that shares the pod's network namespace.

Q13. What is the difference between a ReplicaSet and a Deployment? A ReplicaSet's only job is to ensure a specified number of identical pod replicas are running at all times. A Deployment sits on top of a ReplicaSet and adds rolling updates, rollbacks, and revision history, in practice you almost never create a ReplicaSet directly, you create a Deployment.

Q14. How does a Deployment perform a rolling update? It creates a new ReplicaSet with the updated pod template, then gradually scales up the new ReplicaSet while scaling down the old one, controlled by maxSurge and maxUnavailable settings, ensuring zero downtime during the update.

```
kubectl set image deployment/myapp myapp=myapp:v2  
kubectl rollout status deployment/myapp
```

Q15. How do you rollback a deployment?

```
kubectl rollout undo deployment/myapp  
kubectl rollout undo deployment/myapp --to-revision=2  
kubectl rollout history deployment/myapp
```

Kubernetes keeps revision history of ReplicaSets, allowing rollback to any previous revision.

Q16. What is the difference between Deployment and StatefulSet? A Deployment treats all pod replicas as identical and interchangeable, with random names and no stable storage guarantee. A StatefulSet gives each pod a stable, unique network identity (like app-0, app-1) and stable persistent storage tied to that specific pod, needed for databases and other stateful workloads. StatefulSet is ideal for Active-Passive deployment.

Q17.

I want to deploy Fluentd or Node Exporter. Which k8s object should I use?

What is a DaemonSet and when would you use it? A DaemonSet ensures exactly one copy of a pod runs on every (or a selected subset of) node in the cluster. Common use cases: log collectors (Fluentd), monitoring agents (Node Exporter), or network plugins that must run on every node.

Q18. What is a Job and CronJob in Kubernetes? A Job runs a pod to completion for a one-off task (like a data migration script) and ensures it completes successfully, retrying on failure. A CronJob schedules Jobs to run repeatedly on a time-based schedule, similar to a Linux cron entry.

Q19. What is the difference between kubectl apply and kubectl create? kubectl create creates a new resource and fails if it already exists. kubectl apply is declarative, it creates the resource if it doesn't exist, or patches/updates it to match the YAML if it does, making it the preferred approach for GitOps and repeatable deployments. You may get a warning while attempting kubectl apply on a resource created using kubectl create.

Q20. What happens when a Pod's container crashes, how does Kubernetes handle it? The kubelet restarts the container according to the pod's restartPolicy (Always, OnFailure, or Never). Repeated crashes trigger an exponential backoff delay, which is what causes the well-known CrashLoopBackOff status.

Section 3: Services, Networking, and Ingress

Q21. What is a Kubernetes Service and why is it needed? Pods are ephemeral, they get new IP addresses every time they are recreated. A Service provides a stable virtual IP and DNS name that load-balances traffic across a dynamic set of pods matched by a label selector, so other components never need to track individual pod IPs.

Q22. What are the different types of Services?

- **ClusterIP** (default): accessible only inside the cluster.
- **NodePort**: exposes the service on a static port on every node's IP, accessible externally.
- **LoadBalancer**: provisions an external cloud load balancer (AWS ELB, GCP LB) pointing to the service.
- **ExternalName**: maps a service to an external DNS name, no proxying/load bala involved.

Q23. What is the difference between ClusterIP and NodePort? ClusterIP is only reachable from within the cluster's internal network. NodePort additionally opens a port (30000-32767 range) on every node's external IP, making the service reachable from outside the cluster without a cloud load balancer.

Q24. How does Service discovery work in Kubernetes? Kubernetes runs an internal DNS service (CoreDNS). Every Service automatically gets a DNS name like my-service.my-namespace.svc.cluster.local, and pods can reach other services simply by using that DNS name instead of hardcoded IPs.

Q25. What is an Ingress and how is it different from a Service of type LoadBalancer? A LoadBalancer service provisions one external load balancer per service, expensive at scale. An Ingress is a single entry point that routes HTTP/HTTPS traffic to multiple services based on hostname or URL path rules, requiring only one load balancer in front of the Ingress Controller.

Q26. What is an Ingress Controller? Ingress resources are just rules, they do nothing on their own. An Ingress Controller (like NGINX Ingress Controller, Traefik, or HAProxy) is the actual component running in the cluster that reads Ingress rules and implements the routing logic.

Q27. How does kube-proxy work? kube-proxy watches the API server for Service and Endpoint changes and programs networking rules on each node, traditionally using iptables (rule-based packet matching) or the more performant IPVS mode, to transparently load-balance traffic to the correct backend pods.

Q28. What is a headless service and when is it used? A headless service (clusterIP: None) does not get a single virtual IP, instead DNS returns the individual IPs of all matching pods directly. This is used when clients need to talk to a specific pod, common with StatefulSets like Kafka or Cassandra.

Q29. How do pods communicate with each other across nodes? Kubernetes requires a flat pod network where every pod can reach every other pod's IP directly, without NAT, regardless of which node they're on. This is implemented by a CNI (Container Network

Interface) plugin like Calico, Flannel, Weave, or Cilium.

Section 4: ConfigMaps, Secrets, and Environment Config

Q30. What is a ConfigMap and why use it? A ConfigMap stores non-sensitive configuration data (key-value pairs) separately from the application image, so you can change configuration without rebuilding the container image.

Q31. What is a Secret and how is it different from a ConfigMap? A Secret is meant for sensitive data (passwords, API keys, tokens). Structurally it's similar to a ConfigMap, but values are base64-encoded (not encrypted by default) and Kubernetes handles Secrets with slightly more care, for example not printing values in `kubectl describe` by default.

Q32. How are Secrets stored, are they encrypted by default? By default, Secrets are only base64-encoded, NOT encrypted, in etcd, base64 is trivially reversible, so this is not real security. For actual encryption you must enable Encryption at Rest for etcd or use an external secret manager like HashiCorp Vault or AWS Secrets Manager.

Q33. How do you inject a ConfigMap into a pod? Two ways: as environment variables using `envFrom` or `env.valueFrom.configMapKeyRef`, or as files mounted into the container's filesystem using a volume of type `configMap`.

Q34. What happens if you update a ConfigMap that's mounted as a volume? If mounted as a volume, the file inside the pod eventually updates automatically (after a sync delay, typically under a minute) without restarting the pod. If injected as an environment variable, the pod does NOT see the change until it is restarted, since env vars are set only once at container start.

Q35. How do you create a secret from literal values vs from a file?
`kubectl create secret generic mysecret --from-literal=username=admin --from-literal=password=Pass123`
`kubectl create secret generic mysecret --from-file=./credentials.txt`

Q36. What is the use of imagePullSecrets? It's a Secret of

type docker-registry used to authenticate the kubelet when pulling container images from a private container registry, referenced in the pod spec so Kubernetes can pull images that aren't publicly available.

Section 5: Volumes and Storage

Q37. Why do containers need volumes, what problem do they solve? A container's filesystem is ephemeral, if the container crashes or restarts, any data written inside it is lost. Volumes provide storage that exists independently of the container lifecycle, so data can persist across restarts.

Q38. What is the difference between a Volume and a PersistentVolume (PV)? A regular Volume is tied to the lifecycle of the Pod that defines it, it disappears when the pod is deleted (except a few types). A PersistentVolume is a cluster-level storage resource, provisioned independently of any specific pod, that outlives individual pods and can be reused.

Q39. What is a PersistentVolumeClaim (PVC)? A PVC is a request for storage made by a user/application/service, specifying size and access mode. Kubernetes binds the PVC to a matching PV, the application only interacts with the PVC, never directly with the underlying storage details.

Q40. What is a StorageClass and dynamic provisioning? A StorageClass defines a "class" of storage (like SSD-backed, HDD-backed, or cloud-specific storage) along with a provisioner. When a PVC requests a StorageClass, the provisioner automatically creates a matching PV on demand, this is dynamic provisioning, eliminating the need to manually pre-create PVs.

Q41. What is the difference between emptyDir and hostPath volumes? emptyDir creates a temporary directory that exists only as long as the pod exists on that node, shared between containers in the pod, and is wiped on pod deletion. hostPath mounts a specific path from the underlying node's filesystem directly into the pod, which is generally discouraged in production due to portability and security risks.

Q42. What are access modes in PV?

- **ReadWriteOnce (RWO):** mounted read-write by a single

node.

- **ReadOnlyMany (ROX):** mounted read-only by many nodes.
- **ReadWriteMany (RWX):** mounted read-write by many nodes simultaneously (needs a storage backend like NFS or EFS that supports this).

Q43. What happens to data in a PVC when a Pod is deleted? The data survives, because the PVC and its bound PV exist independently of the Pod. Whether the underlying data survives PVC deletion too depends on the PV's reclaimPolicy (Retain, Delete, or Recycle).

Section 6: Namespaces, Labels, Selectors, and Annotations

Q44. What is a Namespace and why use it? A Namespace is a way to logically partition a single cluster into multiple virtual clusters, commonly used to separate environments (dev, staging, prod) or teams, providing scope for resource names and enabling quota/RBAC boundaries.

Q45. What is the difference between Labels and Annotations? Labels are key-value pairs used to identify and select objects (for example, Services use label selectors to find their pods). Annotations are also key-value pairs but are meant for non-identifying metadata, like build version, contact info, or tool-specific configuration, and are not used for selection.

Q46. What are selectors and how do Services use them? A selector is a query using labels to filter a set of objects. A Service defines a selector field matching pod labels, and Kubernetes continuously updates the Service's Endpoints object with the IPs of all currently matching, ready pods.

Q47. How do you restrict resource usage per namespace? A **ResourceQuota** limits the total amount of CPU, memory, or object count (like max number of pods) allowed within a namespace. A **LimitRange** sets default and max/min resource requests/limits for individual pods/containers within that namespace.

Q48. What are the default namespaces in Kubernetes? default (for objects with no namespace specified), kube-system (control plane and Kubernetes system components), kube-public (readable by all, used for cluster info), and kube-node-lease (node heartbeat lease

objects).

Section 7: Scheduling, Affinity, Taints, and Tolerations

Q49. How does the Kubernetes scheduler decide which node to place a pod on? It runs a two-step process: filtering (eliminate nodes that don't meet requirements like insufficient resources, taints, or affinity mismatches) and scoring (rank remaining eligible nodes based on criteria like resource balance), then picks the highest-scoring node.

Q50. What are Node Affinity and Pod Affinity/Anti-Affinity? Node Affinity attracts pods to nodes matching certain labels (e.g., only schedule on nodes labeled `disktype=ssd`). Pod Affinity/Anti-Affinity controls placement relative to OTHER pods, for example, ensuring replicas of the same app are spread across different nodes (anti-affinity) for high availability, or co-locating related pods (affinity) for low latency.

Q51. What are Taints and Tolerations, give a real world use case? A Taint is applied to a Node to repel pods that don't explicitly tolerate it. A Toleration is applied to a Pod, allowing it to be scheduled on a tainted node. Real world use case: dedicating GPU nodes only to ML workloads by tainting GPU nodes and adding a matching toleration only to ML pod specs.

Q52. What is the difference between Taints and NodeSelectors? NodeSelector is a pull mechanism, a pod requests to be placed on nodes with specific labels. Taints are a push mechanism, a node repels pods unless they explicitly tolerate the taint, this is used for exclusion rather than simple targeting.

Q53. What is a nodeName field and when would you use it directly? Setting nodeName in a pod spec bypasses the scheduler entirely and forces the pod onto a specific named node. It's rarely used in production (loses scheduling flexibility and failover), mainly useful for debugging or very specific hardware-pinning needs.

Q54. What are resource requests and limits, how do they affect scheduling? requests is the guaranteed minimum amount of CPU/memory a container needs, used by the scheduler to decide node placement. limits is the maximum a container is allowed to consume,

enforced at runtime. Based on how requests/limits are set, pods get a QoS class: Guaranteed (requests = limits), Burstable (requests set but less than limits), or BestEffort (neither set), which also determines eviction priority under node pressure.

Q55. What is Pod Priority and Preemption? PriorityClass assigns a priority value to pods. If a high-priority pod cannot be scheduled due to insufficient resources, the scheduler can preempt (evict) lower-priority pods on a node to make room for it.

Section 8: RBAC, Security, and ServiceAccounts

Q56. What is RBAC in Kubernetes? Role-Based Access Control is the authorization mechanism that determines what actions (verbs like get, list, create, delete) a user or service account can perform on which resources (pods, secrets, deployments) within the cluster.

Q57. What is the difference between a Role and a ClusterRole? A Role's permissions are scoped to a single namespace. A ClusterRole's permissions apply cluster-wide, across all namespaces, or to cluster-scoped resources like Nodes or PersistentVolumes.

Q58. What is a RoleBinding vs ClusterRoleBinding? A RoleBinding grants the permissions defined in a Role (or ClusterRole) to a user/group/service account within a specific namespace. A ClusterRoleBinding grants those permissions cluster-wide, across all namespaces.

Q59. What is a ServiceAccount and how is it different from a User? A ServiceAccount is an identity used by processes running INSIDE pods (like an application calling the Kubernetes API). A regular User represents a human or external system authenticating from outside the cluster, Kubernetes itself doesn't manage User objects directly, it relies on external identity providers (certificates, OIDC, etc).

Q60. How do you restrict a pod's Linux capabilities? Using a securityContext, you can set runAsNonRoot: true, drop Linux capabilities (capabilities.drop: ["ALL"]), set readOnlyRootFilesystem: true, and prevent privilege escalation (allowPrivilegeEscalation: false), all reducing the blast radius if a container is compromised.

Q61. What is a Network Policy and why is it needed? By default, all pods in a cluster can talk to all other pods freely. A NetworkPolicy is a firewall-like rule set (enforced by the CNI plugin) that restricts which pods can communicate with which other pods, based on labels, namespaces, and ports, essential for zero-trust security in multi-tenant clusters.

Q62. What is Pod Security Standards / Pod Security

Admission? It's the built-in mechanism (replacing the older deprecated PodSecurityPolicy) that enforces security profiles at the namespace level, three levels exist: Privileged, Baseline, and Restricted, controlling things like whether pods can run as root, use host networking, or mount host paths.

Section 9: Health Checks and Probes

Q63. What are Liveness, Readiness, and Startup probes, and how do they differ?

- **Liveness probe:** checks if the container is still alive/healthy, if it fails, Kubernetes restarts the container.
- **Readiness probe:** checks if the container is ready to accept traffic, if it fails, the pod is removed from Service endpoints (not restarted).
- **Startup probe:** used for slow-starting applications, disables liveness/readiness checks until the app has successfully started, preventing premature restarts.

Q64. What happens if a Readiness probe fails vs a Liveness probe fails? If Readiness fails, the pod stays running but is temporarily removed from the Service's load-balancing rotation until it passes again, no restart happens. If Liveness fails, Kubernetes kills and restarts the container entirely.

Q65. How does Kubernetes achieve self-healing? Through multiple mechanisms working together: the kubelet restarts crashed containers per restartPolicy, the ReplicaSet controller recreates pods if the total count drops below desired, and the Node controller reschedules pods from nodes that go NotReady.

Q66. What is the difference between restartPolicy:

Always, OnFailure, and Never? Always restarts the container

regardless of exit code (default for Deployments). OnFailure restarts only if the container exits with a non-zero (error) exit code, commonly used for Jobs. Never never restarts automatically, also used with Jobs when you want to see exact failure state.

Q67. How would you debug a pod stuck in Running state but not receiving traffic? Check if the Readiness probe is failing (kubectl describe pod), check if the Service selector actually matches the pod's labels, and verify the Endpoints object actually lists the pod's IP (kubectl get endpoints).

Q68. What is a Startup probe used for, why was it introduced? It was introduced because some legacy applications take a long time to boot up (JVM apps, for example), and a Liveness probe firing too early during a slow startup would cause an endless restart loop. The Startup probe gives the app a grace period before liveness/readiness checks kick in.

Section 10: Autoscaling

Q69. What is Horizontal Pod Autoscaler (HPA)? HPA automatically increases or decreases the NUMBER of pod replicas in a Deployment/StatefulSet based on observed metrics like CPU or memory utilization, keeping the application responsive under varying load.

Q70. What metrics can HPA scale on? By default, CPU and memory (via metrics-server). It can also scale on custom metrics (application-specific, like queue length) or external metrics (from outside the cluster, like a cloud monitoring metric), using the Custom Metrics API or External Metrics API.

Q71. What is Vertical Pod Autoscaler (VPA) and how is it different from HPA? VPA automatically adjusts the CPU/memory REQUESTS AND LIMITS of individual pod containers based on historical usage, instead of adding more replicas. HPA scales OUT (more pods), VPA scales UP (bigger pods).

Q72. What is Cluster Autoscaler? It automatically adds or removes NODES from the cluster based on whether pending pods can't be scheduled due to insufficient resources (scale up) or nodes are underutilized for a sustained period (scale down), working at the infrastructure level, complementing HPA/VPA which work at the pod

level.

Q73. Can HPA and VPA be used together, any caveats? Not straightforwardly on the same metric (both trying to react to CPU usage can conflict, causing thrashing). A common pattern is using VPA for right-sizing memory/CPU requests while HPA scales on a different metric like custom application metrics.

Q74. How would you set up HPA based on custom metrics? You need a metrics pipeline (like Prometheus Adapter) exposing the custom metric through the Kubernetes Custom Metrics API, then reference that metric in the HPA spec's metrics field with type: Pods or type: Object, instead of the default Resource type.

Section 11: Helm and Kustomize

Q75. What is Helm and why use it? Helm is the package manager for Kubernetes. It lets you bundle a set of related YAML manifests (Deployments, Services, ConfigMaps) into a reusable, versioned, parameterized package called a Chart, instead of managing dozens of raw YAML files manually.

Q76. What is a Helm Chart, explain its structure? A Chart is a directory with:

- Chart.yaml: metadata (name, version).
- values.yaml: default configurable values.
- templates/: Kubernetes manifest templates using Go templating, referencing values from values.yaml.
- charts/: any dependent sub-charts.

Q77. What is the difference between helm install and helm upgrade? helm install deploys a brand-new release of a chart. helm upgrade updates an existing release with new values or a new chart version, while keeping revision history for rollback.

Q78. What are Helm values.yaml and templates used for? values.yaml holds default configuration (like replica count, image tag, resource limits). Templates reference these values using {{ .Values.xyz }} syntax, letting the same chart be reused across environments just by overriding values (-f custom-values.yaml or --set key=value).

Q79. What is Kustomize and how is it different from

Helm? Kustomize is a native Kubernetes configuration management tool (built into kubectl via kubectl apply -k) that lets you customize plain YAML using overlays and patches, WITHOUT templating syntax. Helm uses a templating language and packages things as versioned charts, Kustomize instead layers "base" and "overlay" YAML directly, many consider it simpler for straightforward environment-specific tweaks.

Q80. How do you rollback a Helm release?

```
helm history myrelease  
helm rollback myrelease 2
```

This reverts the release to a previous stored revision.

Section 12: Troubleshooting and Debugging Scenarios

Q81. A pod is stuck in CrashLoopBackOff, how do you debug it?

```
kubectl describe pod <pod-name>  
kubectl logs <pod-name>  
kubectl logs <pod-name> --previous
```

Check the previous container's logs (it has already crashed and restarted), look for application errors, missing environment variables, or failed dependency connections. Also check the exit code in the describe output, it often points to the exact cause (OOMKilled, error code 1, etc).

Q82. A pod is stuck in Pending state, what could be the

reasons? Common causes: insufficient CPU/memory on all nodes to satisfy requests, no node matches the pod's nodeSelector/affinity rules, an unresolved taint with no matching toleration, or a PVC that cannot be bound to any PV. kubectl describe pod will show scheduling failure events explaining exactly which condition failed.

Q83. A pod shows ImagePullBackOff, how do you fix it?

Check the image name/tag for typos, verify the image actually exists in the registry, and confirm imagePullSecrets are correctly configured if pulling from a private registry.
kubectl describe pod <pod-name>

This shows the exact pull error (auth failure, image not found, network timeout).

Q84. Service is not routing traffic to pods, how do you debug?

```
kubectl get endpoints <service-name>
```

```
kubectl describe svc <service-name>
```

If Endpoints is empty, the Service's label selector doesn't match any pod's labels, or the matching pods are failing their Readiness probe. Also confirm the target port in the Service matches the container's actual listening port.

Q85. Node shows NotReady status, how do you troubleshoot?

```
kubectl describe node <node-name>
```

```
kubectl get events --field-selector involvedObject.name=<node-name>
```

Check kubelet status on the node itself (systemctl status kubelet), disk pressure, memory pressure, or network plugin failures, all commonly reported in node Conditions.

Q86. A deployment rollout is stuck, how do you investigate?

```
kubectl rollout status deployment/myapp
```

```
kubectl describe deployment myapp
```

```
kubectl get rs
```

Look at the new ReplicaSet's pods, they're likely stuck in Pending or CrashLoopBackOff, trace back using the earlier debugging steps for those specific pod failure types.

Q87. How do you check why a pod was OOMKilled?

```
kubectl describe pod <pod-name>
```

Look for Last State: Terminated, Reason: OOMKilled under container status. Fix by increasing the memory limit, or investigating a memory leak in the application, and cross-check actual usage with `kubectl top pod`.

Q88. kubectl exec into a pod fails, what could be wrong? The pod might not be Running (check `kubectl get pod`), the container image might lack a shell (common with minimal/distroless

images, sh or bash doesn't exist), or you might lack RBAC permission (pods/exec verb) to exec into pods in that namespace.

Q89. DNS resolution fails inside a pod, how do you debug?

```
kubectrl exec -it <pod-name> -- nslookup kubernetes.default
```

```
kubectrl get pods -n kube-system -l k8s-app=kube-dns
```

```
kubectrl logs -n kube-system <coredns-pod>
```

Check that CoreDNS pods are running and healthy, verify the pod's /etc/resolv.conf points to the cluster DNS service IP, and confirm no restrictive NetworkPolicy is blocking traffic to CoreDNS on port 53.

Q90. How would you debug high latency in a microservice

running on Kubernetes? Check resource throttling first (kubectrl top pod, compare against CPU limits, since CPU throttling silently causes latency spikes). Check readiness/liveness probe timing and restart frequency. Check if HPA is scaling reactively too slowly under sudden load. Finally check downstream dependencies (database, other services) and network policies or service mesh sidecar overhead as contributing factors.